Lesson 13 Performing Color Detection with OpenCV

In this tutorial we will learn how to detect colors using OpenCV.

13.1 Color Detection & Color Space

- Refer to Lesson 10 for preparations and running operations for OpenCV function
- The code of the color detection function to be explained here is included in the camera_opencv.py file which was created in the same folder with the app.py and base_camera.py files.
- For safety reason, in this lesson we will not control the motor or serve to move but just obtain the results calculated with OpenCV.
- HSV color space is adopted for color detection with OpenCV. Before learning the code, first you need to know what color space is and why HSV is used here instead of the more commonly seen RGB color space.

Color Space

Color space refers to a method of organizing colors. With the help of color space and tests of physical devices, we can get a certain analog and digital representation of colors. Color space can be defined by random colors. For example, Pantone series uses a group of specific colors as sample, and define each color with a name and code. Or, define it mathematically like Adobe RGB, sRGB, etc.

RGB Color Space

RGB uses additive colors as it describes a specific proportion by which various kinds of "light" produces colors. Starting from black, light keeps mixing to generate different colors. RGB

means the value of red, green, and blue; RGBA produces a transparent color based on RGB with the alpha channel.

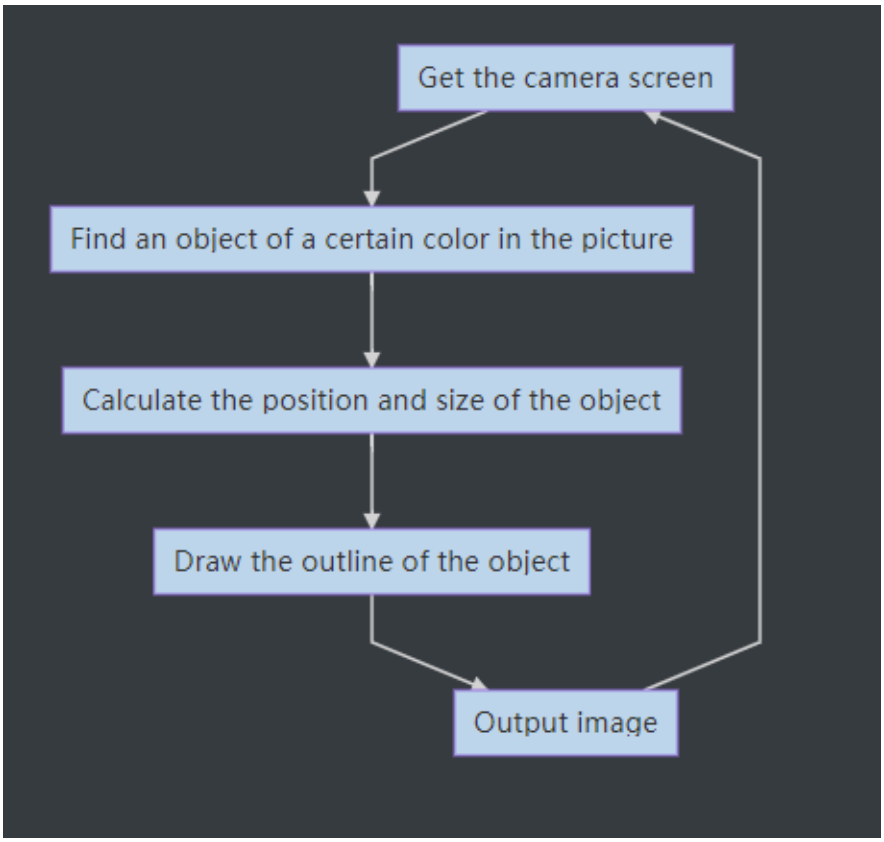
Common color spaces based on the RGB mode include sRGB, Adobe RGB and Adobe Wide Gamut RGB.

HSV Color Space

HSV, or hue, saturation, and value, also referred to as HSB (B for brightness), is commonly used by artists because it's more understandable to describe color by hue and saturation compared with terms of additive or subtractive color mixing. HSV is a variant of RGB color space and closely related in content and color standards as it derives from RGB.

HSV color space is used for color detection with OpenCV since it's less effected by ambient light and brings more accurate detection results. Besides, it's easy to define color range with HSV. During color detection, the code is to recognize some range of colors instead of some color. Therefore, the HSV color space is used for color detection since it is similar to the human perception of color.

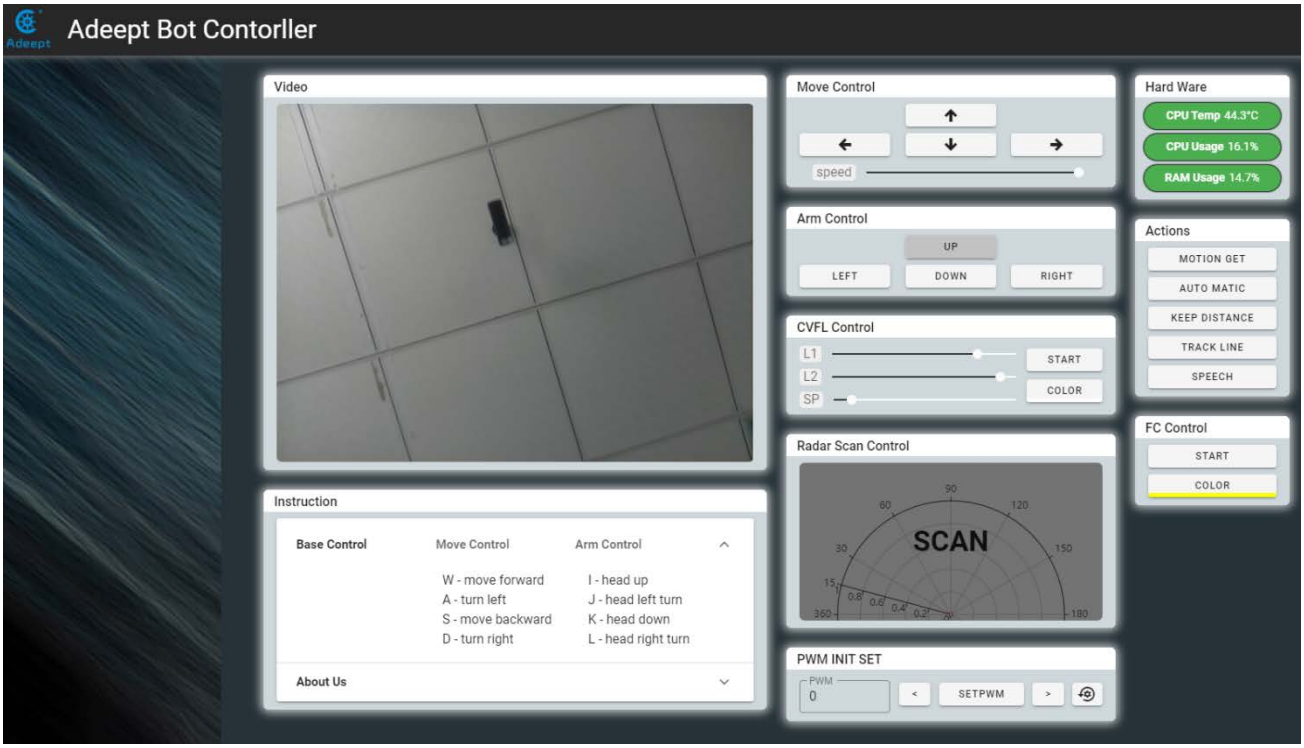
Here we use this function to aim the camera at a target of a specific color with the function. The process is as follows:



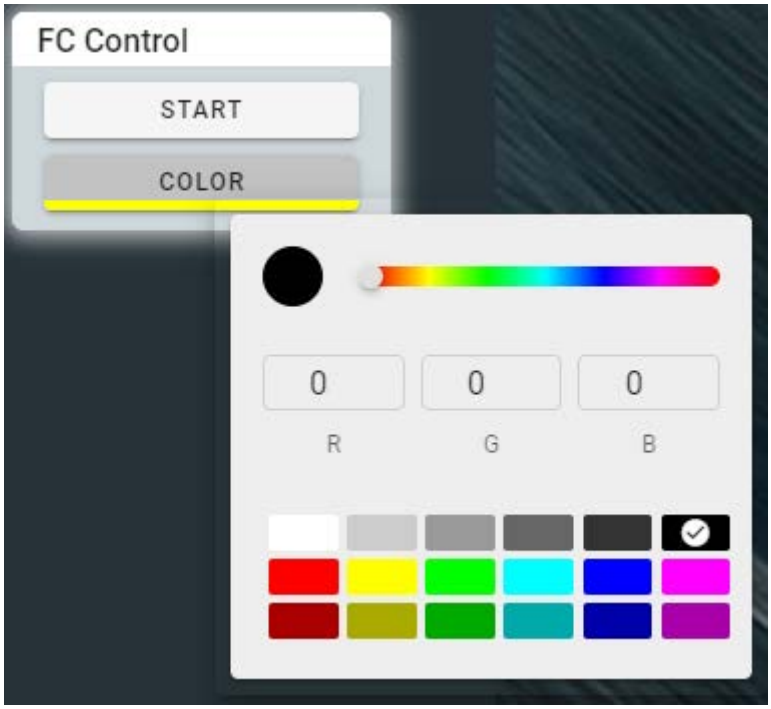
13.2 Running Color Detection

Power on the robot; it may take 30s-50s (it will automatically run the webServer.py when booting up).

After booting up, open a Google Chrome web browser on your mobile or computer, enter the address of your Raspberry Pi and ":port number". For example, 192.168.3.44:5000, and then the web controller will be shown on the browser.



1. Click the COLOR button in the FC Control section, select the color to be detected (yellow is selected by default).



2. After selecting the color, click START to start the color detection.

3. Click START again to end the detection.

13.3 Main Program

Refer to the camera_opencv.py file for more details.

```
import os
import cv2
from base_camera import BaseCamera
import numpy as np

"""
Configure target color and HSV color space
"""
colorUpper = np.array([44, 255, 255])
colorLower = np.array([24, 100, 100])

font = cv2.FONT_HERSHEY_SIMPLEX
```

```
class Camera(BaseCamera):
    video_source = 0

    def __init__(self):
        if os.environ.get('OPENCV_CAMERA_SOURCE'):
            Camera.set_video_source(int(os.environ['OPENCV_CAMERA_SOURCE']))
        super(Camera, self).__init__()

    @staticmethod
    def set_video_source(source):
        Camera.video_source = source

    @staticmethod
    def frames():
        camera = cv2.VideoCapture(Camera.video_source)
        if not camera.isOpened():
            raise RuntimeError('Could not start camera.')

        while True:
            # read current frame
            _, img = camera.read() #Obtain images captured by the camera

            hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)  #Transfrom the images to HSV color space
            mask = cv2.inRange(hsv, colorLower, colorUpper) #Loop to detect the color based on the target color range in the HSV color space, and turn the color blocks into masks
            mask = cv2.erode(mask, None, iterations=2) #Erode and diminish the small masks (hot pixels) in the image (eliminate small color blocks or hot pixels)
            mask = cv2.dilate(mask, None, iterations=2) #Dilate, to resize the large masks eroded in the previous line to the original
            cnts = cv2.findContours(mask.copy(), cv2.RETR_EXTERNAL,
                cv2.CHAIN_APPROX_SIMPLE)[-2]          #Find masks in the image
            center = None
            if len(cnts) > 0:   #If the number of masks is more than 1,
                """
                Find the coordinate of the center and size of the target color object in the image
                '''
                c = max(cnts, key=cv2.contourArea)
                ((box_x, box_y), radius) = cv2.minEnclosingCircle(c)
                M = cv2.moments(c)
                center = (int(M["m10"] / M["m00"]), int(M["m01"] / M["m00"]))
                X = int(box_x)
```

```
            Y = int(box_y)
            """
            Obtain and output the coordinate of the center of the target color object
            '''
            print('Target color object detected')
            print('X:%d'%X)
            print('Y:%d'%Y)
            print('-------')

            """
            Show the text "Target Detected" in the image
            '''
            cv2.putText(img,'Target Detected',(40,60), font, 0.5,(255,255,255),1,cv2.LINE_AA)
            """
            Mark the target with a frame
            '''
            cv2.rectangle(img,(int(box_x-radius),int(box_y+radius)),
                    (int(box_x+radius),int(box_y-radius)),(255,255,255),1)
        else:
            cv2.putText(img,'Target Detecting',(40,60), font, 0.5,(255,255,255),1,cv2.LINE_AA)
            print('No target color object detected')

        # encode as a jpeg image and return it
        yield cv2.imencode('.jpg', img)[1].tobytes()
```

You can select the color to be detected by changing the colorUpper and colorLower. Note that the H value of general HSV color space is 0-360, but in OpenCV it's 0-180.

13.4 HSV Color Component Range in OpenCV

Generally the H (hue) value of HSV color space ranges from 0-360, yet 0-180 in OpenCV.

HSV\Color	Black	gray	white	red		orange	yellow	green	cyan	blue	purple
H_min	0	0	0	0	156	11	26	35	78	100	125
H_max	180	180	180	10	180	25	34	77	99	124	155

S_min	0	0	0	43	43	43	43	43	43	43
S_max	255	43	30	255	255	255	255	255	255	255
V_min	0	46	221	46	46	46	46	46	46	46
V_max	46	220	255	255	255	255	255	255	255	255